MLSysBook RAG Learning Companion

A retrieval-augmented learning companion for the two-volume textbook ***Machine Learning Systems*** **by Vijay Janapa Reddi** (Harvard; source, CC BY-NC-SA 4.0). Ask a question, get a grounded answer with inline citations to specific chapters and sections — and see every retrieval score and latency stage that produced it.

The meta-angle is deliberate: **an ML-systems engineering project about an ML-systems textbook.** The stack demonstrates the lifecycle the book teaches — ingestion, retrieval, serving, evaluation, monitoring, inference optimisation — with every claim backed by a measurement.

> **Attribution.** Book content © Vijay Janapa Reddi, licensed CC BY-NC-SA 4.0. This project's code is MIT. No book text is committed anywhere in this repository — you build your own index (see LICENSING.md).

What it is

```
                                ┌──────────────────────────────────────┐
                                │              k3s cluster             │
 ┌──────────┐    HTTPS/SSE   ┌──────────┐  ┌──────────┐  ┌────────────┐ │
 │ Frontend │                │ Gateway  │─▶│ Embedder │  │  Reranker  │ │
 │ (React)  │  ───────────▶  │(FastAPI) │  │ (bge-m3) │  │(bge-r-v2)  │ │
 └──────────┘                │ HPA 1-5  │  └──────────┘  └────────────┘ │
                             └──────────┘       │              ▲        │
                                │               ▼              │        │
                                │          ┌────────────┐      │        │
                                ├─────────▶│  Postgres  │──────┘        │
                                │          │ + pgvector │ hybrid top-30 │
                                │          └────────────┘  → rerank top-5│
                                ▼                                       │
                             ┌──────────┐   ┌────────────┐              │
                             │   vLLM   │   │ Prometheus │              │
                             │ Qwen3.8- │   │ + Grafana  │              │
                             │ 27B W4A16│   │ + DCGM     │              │
                             └──────────┘   └────────────┘              │
                                └───────────────────────────────────────┘
```

Request lifecycle: question → embed (bge-m3) → hybrid retrieval in one SQL round trip (dense HNSW + Postgres FTS, Reciprocal Rank Fusion k=60, top-30) → cross-encoder rerank (bge-reranker-v2-m3, top-5) → numbered context blocks → streaming generation (Qwen3.8-27B W4A16 on vLLM, one RTX 3090 Ti) → SSE: a `citations` event *before the first token*, then `token` events, then `done` with per-stage latencies and token usage. Every stage is a Prometheus histogram and a per-query log row.

Headline pieces

- A **four-model ablation** where each pairwise comparison varies exactly one factor with the model family held constant: size (27B vs 9B), architecture (dense vs MoE), serving engine (vLLM vs llama.cpp + MTP).
- A **27B hybrid-attention model on consumer Ampere** with the batching/KV-memory story that architecture enables.
- A **real evaluation harness**: hand-verified golden set (the harness refuses to run on an unverified one), retrieval recall/MRR for dense → hybrid → hybrid+rerank, a validated LLM-as-judge, and abstention scoring.
- **Kubernetes on bare metal with an honest scaling story:** HPA on the stateless tier, and a written analysis of why horizontal scaling is the wrong axis for a single-GPU LLM tier (docs/writeups/scaling.md).

Status

Milestone	State	Evidence
M0 scaffold, CI, content guard	✅	`make ci`; `.github/workflows/ci.yml`; `scripts/guard_content.py`
M1 ingestion	✅ measured	47 chapters → 2,815 chunks, idempotent re-run (m2-retrieval.md)
M2 index + hybrid retrieval + reranker	✅ measured	GPU index 28 s; CPU query-embed p50 71 ms; smoke test
M3 vLLM bring-up (risk milestone)	⏳ not yet run on the GPU	procedure + tables in m3-baseline.md
M4 gateway (SSE, shim, metrics, demo profile)	✅	34 tests incl. the official `openai` client; overhead ~5 ms
M5 eval harness + golden set	🔧 harness done; golden set needs the owner's verification pass	`make golden-generate` → `make golden-verify` → `make eval`
M6 docker-compose	🔧 written, not yet executed on this host	DEVIATIONS.md #4
M7 k8s + monitoring	🔧 manifests, dashboards, alerts, CronJob written; cluster not yet brought up	`k8s/`, `make hpa-demo`
M8 benchmarks + ablations	🔧 harness + sweep configs done; runs pending M3	`bench/`, benchmark-report.md
M9 frontend	✅	Lighthouse 100/100/96 (lighthouse.md)
M10 hosted demo	🔧 demo profile + cost controls tested; Supabase/Fly/Pages deploy pending credentials	`gateway/tests/test_api.py::test_demo_profile_rate_limit_and_budget`
M11 writeups	🔧 this README, LICENSING, runbook, scaling;	`docs/`

Milestone	State	Evidence
	benchmark report awaits numbers	

Quickstart (5 commands, no Docker needed)

```
make setup && make setup-models                 # uv workspace + frontend deps (+ torch cu128, models)
make db-up && export DATABASE_URL='...'         # unprivileged Postgres+pgvector under data/pg
make ingest && make index                       # fetch book @ pinned SHA → 2,815 chunks → bge-m3 HNSW
EMBEDDER_MODE=gpu make embedder & RERANKER_MODE=gpu make reranker & make vllm &
make gateway & (cd frontend && pnpm dev)        # http://localhost:5173
```

```
curl -N localhost:8000/api/ask -H 'content-type: application/json' \
     -d '{"question":"Why is the KV cache a bottleneck for LLM inference?"}'
```

The full, tested runbook — including the compose and k3s paths — is docs/RUN_IT_YOURSELF.md. No GPU? `LLM_MODEL=fake make gateway` exercises the entire pipeline with a stand-in generator, or point `LLM_BASE_URL` at any OpenAI-compatible server.

The hosted demo is not the system

The public demo (Cloudflare Pages → same gateway code with `PROFILE=demo` → Supabase pgvector → **Claude Haiku** for generation) is an accessibility layer so you can try the retrieval and citation contract without a GPU. It is rate-limited (10 questions/day/IP) with a hard daily budget stop. The k3s/vLLM local stack above is the actual project; the `docker compose up` path and the demo video are the way to see it.

Repository map

Path	What
`ingest/`	fetch @ pinned SHA, Quarto parser, structure-aware chunker, idempotent loader
`embedder/`, `reranker/`	bge-m3 / bge-reranker-v2-m3 services (gpu · cpu · onnx-int8 modes)
`gateway/`	FastAPI: `/api/ask` SSE, `/v1/chat/completions` shim, coverage, metrics, demo cost controls, `prompts/v1`
`eval/`	golden-set generation + **human verification CLI**, metrics, 3-prompt judge, drift CronJob entrypoint
`bench/`	async load generator, sweep configs (prefix caching, eager, MTP, KV dtype, reasoning), report
`frontend/`	React + TS + Vite + Tailwind: chat w/ citation chips, retrieval inspector, coverage browser, eval dashboard, about
`k8s/`	numbered plain manifests (no Helm/Kustomize), Prometheus rules, 3 Grafana dashboards, HPA, DCGM, drift CronJob
`docker/`	multi-stage Dockerfiles, compose profiles `core · llm · llm-gguf · obs · frontend`
`config/`	`ingest.yaml` (pinned SHA), `models.yaml` (slate), `serving/*.yaml` (exact flags per engine)
`docs/`	runbook, writeups, per-milestone measurements, DEVIATIONS.md

Measured so far

- Chunking: 2,815 chunks, p50 680 tokens in the bge-m3 tokenizer; re-ingest is a content-hash no-op.
- Retrieval: embed 9 ms (GPU) / 71 ms p50 (CPU), hybrid SQL 3–14 ms, rerank 260 ms (GPU, 30 docs).
- Gateway: p50 overhead ≈ 5 ms excluding model + service time; the `openai` client works unmodified against the shim.
- Frontend: Lighthouse performance 100, 61 KB gzipped.
- Everything else — TTFT/tok-per-second vs concurrency, the ablation matrix, lever sweeps, HPA 1 → 4 → 1, dashboards — lands in `docs/benchmarks/` as the GPU milestones run. Placeholders are labelled *pending*; no number is typed by hand.

Out of scope for v1

Fine-tuning, multi-agent designs, conversation memory, user accounts, other sources, Helm/Kustomize, multi-node. Ideas go to FUTURE.md.